

Fregata: Faster Homomorphic Evaluation of AES via TFHE

Benqiang Wei[1,2], Ruida Wang[1,2], Zhihao Li[1,2], Qinju Liu[1,2], and Xianhui Lu[1,2(✉)]

[1] State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
luxianhui@iie.ac.cn

[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

Abstract. Gentry et al. [26] first presented a homomorphic evaluation of the AES-128 based on the BGV scheme, however, it suffered from high evaluation latency. Despite considerable efforts have been directed towards designing FHE-friendly symmetric encryption algorithms, the efficient homomorphic evaluation of the well-studied and standardized AES remains an attractive challenge for researchers in the transciphering community.

In this paper, we present a novel homomorphic evaluation framework based on the TFHE scheme, demonstrating the optimal latency for AES-128 evaluation. Specifically, we propose mixed packing to achieve efficient S-box evaluation and an optimized circuit bootstrapping as a bridge to connect the whole evaluation framework. Furthermore, we show the versatility of our evaluation framework by extending it to other ciphers, such as SM4. To validate the effectiveness of our proposed framework, we conduct implementation experiments, which indicate that the evaluation of AES takes 86 s on a single core, a 3× improvement over the state-of-the-art [39]. Moreover, with a 16-thread parallel implementation, it takes about 9 s. For SM4 evaluation, it takes only 78 s on a single core, about 73× improvement compared to publicly available BGV-based solution [40].

Keywords: TFHE · Transciphering · AES · Circuit bootstrapping

1 Introduction

Fully homomorphic encryption (FHE) enables the computation of arbitrary functions to be performed on encrypted data without decryption. Prior to 2009, FHE had been considered an open problem until Gentry proposed the first feasible fully homomorphic encryption based in ideal lattices [25]. This seminal work starts the prelude to the booming development of fully homomorphic encryption. Some representative schemes include BGV [6], BFV [5,24], CKKS [10], FHEW [23], TFHE [11,12], Final [3] and [30].

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023
E. Athanasopoulos and B. Mennink (Eds.): ISC 2023, LNCS 14411, pp. 392–412, 2023.
https://doi.org/10.1007/978-3-031-49187-0_20

However, the size of homomorphic ciphertext is generally several orders of magnitude larger than the corresponding plaintext, as a result, the data sender would suffer from large ciphertext expansion and communication cost, particularly on the embedded devices with limited bandwidth, memory and computing power. To address this problem, transciphering is firstly proposed by Naehrig et al. [36]. The main idea behind transciphering is that instead of using homomorphic encryption to encrypt data, the client sends the data encrypted by symmetric encryption ($\mathcal{E}$) to the server. Then, the server homomorphically evaluates the symmetric encryption scheme's decryption circuit ($\mathcal{E}^{-1}$) to convert the symmetrically-encrypted data ($\mathcal{E}(m)$) into homomorphic ciphertext. Afterward, the server can proceed to perform the actual function on homomorphic ciphertext. Therefore, the homomorphic evaluation of $\mathcal{E}^{-1}$ becomes a critical problem within the transciphering framework.

The Advanced Encryption Standard (AES) is a widely embraced block encryption standard by the United States federal government, known for its efficiency and prevalent use in securing sensitive information across diverse applications. Therefore, AES stands as one of the top choices for application in the transciphering framework. However, despite its popularity, current research widely acknowledges that AES is not inherently well-suited as an FHE-friendly symmetric encryption scheme. The primary limitation lies in its exceptionally high multiplicative depth, leading to significantly long latency during homomorphic evaluation. To address this issue, some researchers have shifted their focus on designing FHE-friendly symmetric encryption algorithms like LowMC [1], Chaghri [2], Rubato [28], Pasta [20], Elisabeth [16]. However, none of these alternatives have been standardized, and some have been found to be vulnerable to attacks. The efficient homomorphic evaluation of the well-studied and standardized AES remains an attractive challenge for researchers in the transciphering community. Consequently, extensive efforts have been dedicated to improving the homomorphic evaluation latency of AES to make it more amenable to fully homomorphic encryption.

State-of-the-Art. Early in 2012, Gentry et al. [26] presented the first homomorphic evaluation of the AES circuit based on the BGV scheme. The main breakthrough in their work is the homomorphic evaluation of the S-box, which converts the inverse X^{-1} of the finite field $\mathbb{F}_{2^8}$ into the multiplication X^{254}, requiring only three multiplicative depths by leveraging the frobenius automorphism. Overall, the multiplicative depth required to evaluate AES-128 is 40. And they provided two versions of implementation: (1) leveled evaluation version (no bootstrapping): this version demands larger encryption parameter sets but has a relatively shorter latency of about 4 min, resulting in a throughput of 2 s per block. However, it comes with the limitation of not supporting the evaluation of further operations beyond the current computation. (2) bootstrapping version: this version allows for the use of smaller parameters, but the overall evaluation latency increases to up to 18 min due to the use of bootstrapping, resulting in a throughput of 5.8 s per block. Despite the longer delay, this allows further calculations after the homomorphic execution of the AES.

The TFHE scheme supports faster bootstrapping, in particular the introduction of functional bootstrapping [4], which provides a powerful tool to support lookup table calculation for arbitrary functions. Stracovsky et al. [38] reported that they utilized functional bootstrapping technique to evaluate a single block of AES in 4 min. Note that they utilize 16 threads to process 16 bytes in the state matrix in parallel. But they did not provide detailed implementation, only a poster presentation was made at FHE.org in 2022. Recently Trama et al. [39] conducted a comprehensive analysis of a faster AES evaluation implementation based on functional bootstrapping and multi-value bootstrapping techniques. Specifically, instead of directly encrypting a byte, they divide 8-bit into two 4-bit messages to be encrypted, allowing them to use smaller parameters. Thus, for XOR, S-box and multiplication operations on 8-bit message in the encrypted domain, they are transformed into operations between 4-bit messages and evaluated by functional bootstrapping lookup tables. Finally, they evaluated an AES block in 4.5 min using sequential execution on a standard laptop. Indeed, functional bootstrapping is effective for handling S-box lookup table calculation efficiently, but it may be not efficient enough for a substantial number of XOR and multiplication operations. Relying on functional bootstrapping for these operations lead to a reduction in the overall evaluation efficiency. Moreover, the latency for the parallel implementation using 16 cores is about 28 s and still much lower than the amortization time of the BGV-based scheme. An intriguing question arises:

Whether the homomorphic evaluation latency of an AES block based on the TFHE scheme can approach or even reach the same amortization time as that achieved by the BGV-based scheme?

1.1 Contributions

In this paper, we present a novel evaluation framework for faster AES homomorphic evaluation. Specifically, instead of relying on expensive functional bootstrapping, we use inexpensive CMUX gate as the base unit and propose mixed packing technique for efficient S-box evaluation. We also incorporate optimized circuit bootstrapping as a bridge of the framework to speed up the overall evaluation efficiency. We also show that our new evaluation framework can be extended to other Feistel structure ciphers, such as SM4.

Finally, we implemented our proposed framework based on the TFHEpp homomorphic encryption library [34]. Experiment results demonstrate remarkable performance improvements for both AES and SM4 homomorphic evaluation. For AES, the evaluation time of a single block takes only 86 s with a single thread, which is a 3× improvement compared to the state-of-the-art [39]. With 16-thread parallelization, the evaluation time reduces to approximately 9 s, which is very close to the amortization time of the BGV method. For SM4, the evaluation time is about 78 s, a 73× improvement when compared to publicly available BGV-based approach [40].

1.2 Related Works

The homomorphic evaluation of AES has been studied using various homomorphic encryption schemes, in addition to the BGV-based approach presented by Gentry et al. [26]. Cheon et al. [9] evaluated the AES circuit based on vDGHV [18] scheme that supports integer batch processing, and Coron et al. [15] used a variant of vDGHV scheme with the scale-invariant property. Doröz et al. [21] proposed the AES evaluation based on the modified LTV scheme. However, their evaluation efficiency is still unsatisfactory.

Although AES has an algebraic structure that matches well with the plaintext space of some homomorphic encryption schemes, it is not trivial and has great multiplicative depth, resulting very long evaluation time. Therefore, some works attempt to perform the homomorphic evaluation on lightweight block ciphers with low evaluation depth, such as SIMON [31] and Prince [22] algorithm. Moreover, symmetric ciphers with low multiplicative depth are designed for FHE, such as block cipher LowMC [1].

Since stream ciphers only involve XOR operations in the process of encryption and decryption, thus key generation can be done offline in advance, new stream cipher design criteria for FHE is to minimize the multiplicative complexity and depth of the algorithm, such as FLIP [35], Kreyvium [7], Rasta [19] and its variants Dasta [29], Masta [27], Pasta [20], Fasta [14] and Elisabeth [16].

1.3 Organization.

We first review the TFHE scheme and give a brief introduction to different bootstrapping types in Sect. 2. In Sect. 3, we provide a short specification of AES. A novel framework is proposed to evaluate AES in Sect. 4. We demonstrate the scalability of our evaluation framework by giving an efficient evaluation implementation of SM4 in Sect. 5. Implementation results and performance analysis are provided in Sect. 6. Our conclusion is shown in Sect. 7.

2 Preliminaries

2.1 Notations

We denote by λ the security parameter. The set $\{0,1\}$ is written as $\mathbb{B}$. The real torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ is the set of real numbers modulo 1. $\mathfrak{R}$ is the ring $\mathbb{Z}[X]/(X^N+1)$ of integer polynomials modulo X^N+1, and $\mathbb{T}_N[X]$ is $\mathbb{R}[X]/(X^N+1)$ mod 1 of torus polynomials, where N is a power of 2. $\mathbb{B}_N[X]$ denotes the polynomials with binary coefficients. Note that $\mathbb{T}$ is a $\mathbb{Z}$-module and $\mathbb{T}_N[X]$ is a $\mathfrak{R}$-module. We use $< \cdot >$ to denote the inner product.

2.2 The TFHE Scheme

In this subsection, we will review the TFHE scheme. TFHE is a fully homomorphic encryption scheme proposed by Chillotti et al. [11], which is based on the (ring) learning with errors [33,37] problem. There are three main forms of ciphertext in TFHE, which are summarized as follows:

- **TLWE ciphertext:** TLWE encrypts a plaintext value $\mu \in \mathbb{T}$ into a ciphertext $(\vec{a}, b) \in \mathbb{T}^{n+1}$, where $b =< \vec{a}, \vec{s} > +\mu + e$, the vector $\vec{a}$ is uniformly sampled from $\mathbb{T}^n$, the secret key $\vec{s}$ is uniformly sampled from $\mathbb{B}^n$, and the error $e \in \mathbb{T}$ is sampled from a Gaussian distribution with mean 0 and standard deviation σ.
- **TRLWE ciphertext:** TRLWE encrypts a plaintext polynomial $\mu(x) \in \mathbb{T}_N[X]$ into a ciphertext $(\vec{a}, b) \in \mathbb{T}_N[X]^{k+1}$, where $b =< \vec{a}, \vec{s} > +\mu + e$, the vector $\vec{a}$ is uniformly sampled from $\mathbb{T}_N[X]^k$, the secret key $\vec{s}$ is uniformly sampled from $\mathbb{B}_N[X]^k$, and the error $e \in \mathbb{T}_N[X]$ is a polynomial with random coefficients $e_i \in \mathbb{T}$ sampled from a Gaussian distribution with mean 0 and standard deviation σ. If omitted, $k = 1$.
- **TRGSW ciphertext:** TRGSW encrypts the message $\mu \in \mathfrak{R}$ into $C = Z + \mu \cdot G \in \mathcal{M}_{(k+1)l,k+1}(\mathbb{T}_N[X])$, where $Z \in \mathcal{M}_{(k+1)l,k+1}(\mathbb{T}_N[X])$ is matrix such that each line is a random **TRLWE** ciphertext of 0 under the same key, and G is gadget matrix which is used to control the noise propagation.

2.3 Homomorphic Arithmetic of TFHE

In this subsection, we briefly revisit some algorithms in the TFHE scheme.

Controlled Mux Gate. There are two kinds of controlled Mux gates as follows, one built by gate bootstrapping, named bootsMUX and the other by external product, called CMUX.

- $bootsMUX(c, d_0, d_1) = c?d_1 : d_0 = (c \wedge d_1) \oplus ((1 - c) \wedge d_0)$
- $CMUX(c, d_1, d_0) = c \boxdot (d_1 - d_0) + d_0$, where $\boxdot$ denotes the external product TRGSW $\times$ TRLWE $\rightarrow$ TRLWE.

Key Switching. In [12], Chillotti et al. proposed two kinds of Key Switching. The first one is *Public Functional KeySwitching*, which allows switching from TLWE to T(R)LWE samples, such as packing TLWE samples into TRLWE sample or switching secret key. It can also evaluate the public linear function f on the input TLWE samples. The second one is *Private Functional KeySwitching*. The private linear function on the input TLWE samples is evaluated by encoding the secret f into the KeySwitch key.

Blind Rotation. Blind rotation is the core operation of TFHE bootstrapping. It rotates the coefficients of test polynomial blindly using encrypted numbers. The blind rotation operation is mainly constructed by the external product, and its complexity comes from the frequent transformations of FFT and IFFT.

2.4 Bootstrapping Types of TFHE

There are several types of bootstrapping involved in the TFHE scheme, we summarize as follows.

Functional Bootstrapping (FBS). The most important feature of TFHE is the so-called gate bootstrapping. The term gate bootstrapping refers to the fact that every gate evaluation is followed by fast bootstrapping. For example, homomorphic NAND gate is $\text{bootsNAND}(c_1, c_2) = \text{Bootstrap}((\vec{0}, \frac{5}{8}) - c_1 - c_2)$. Functional bootstrapping (or programmable bootstrapping, PBS) was firstly proposed by Boura et al. [4]. The technique is an extension of TFHE gate bootstrapping, which allows to reset the noise to a fixed level and evaluates an arbitrary function on the input at the same time. In more detail, the test polynomial can encode all function values in a discrete manner, then lookup table (LUT) evaluation is performed by using LWE ciphertext as a selector in Blind Rotation.

Multi-Value Bootstrapping (MVBS). Carpov et al. [8] firstly introduced a multi-value bootstrapping technique, which supports the evaluation of multiple functions on the same input ciphertext using only one blind rotation. Firstly, extract a common function v from all functions (TV_{F_i}), then use the input ciphertext to blindly rotate test polynomial v, finally $\frac{TV_{F_i}}{v}$ (with small norm coefficient) is multiplied by accumulator ACC to obtain the functional result respectively.

An alternative multi-value bootstrapping technique, PBSmanyLUT, was proposed by Chillotti et al. [13]. PBSmanyLUT firstly uses modulus switching to set some of the lowest bits of the phase to zero so that all function values can be encoded in a test polynomial. Then only one blind rotation is utilized to obtain all desired function values.

Circuit Bootstrapping (CBS). Circuit bootstrapping, proposed in [12], can convert a TLWE sample with large noise amplitude over binary message space (e.g., amplitude $\frac{1}{4}$ over $\{0, \frac{1}{2}\}$) to a TRGSW sample with a lower noise over the integer message space $\{0, 1\}$, which is 10 times more expensive than the gate bootstrapping, but enables the possibility of fully composable CMUX circuits.

3 Specification of AES

AES-128, being the variant of AES with a 128-bit key, operates on a 128-bit plaintext message (16 bytes), which is represented as a state matrix. The AES encryption process comprises multiple rounds (in this case, 10 rounds for AES-128), each consisting of four main operations: SubBytes, ShiftRows, MixColumns, and AddRoundKey. These operations are applied to the state matrix iteratively to achieve encryption. The state matrix is organized as a 4×4 array of bytes, where each byte represents a single element of the state matrix as follows.

$$\begin{pmatrix} A_0 & A_4 & A_8 & A_{12} \\ A_1 & A_5 & A_9 & A_{13} \\ A_2 & A_6 & A_{10} & A_{14} \\ A_3 & A_7 & A_{11} & A_{15} \end{pmatrix}$$

– **SubBytes:** Also known as S-box, the S-box transforms each element of state matrix non-linearly using a lookup table with special mathematical properties:

$$\begin{pmatrix} B_0 & B_4 & B_8 & B_{12} \\ B_1 & B_5 & B_9 & B_{13} \\ B_2 & B_6 & B_{10} & B_{14} \\ B_3 & B_7 & B_{11} & B_{15} \end{pmatrix} = \begin{pmatrix} S(A_0) & S(A_4) & S(A_8) & S(A_{12}) \\ S(A_1) & S(A_5) & S(A_9) & S(A_{13}) \\ S(A_2) & S(A_6) & S(A_{10}) & S(A_{14}) \\ S(A_3) & S(A_7) & S(A_{11}) & S(A_{15}) \end{pmatrix}$$

– **ShiftRows:** In a 4×4 state matrix, the first row remains unchanged, while the second row undergoes a cyclic shift to the left by one byte. Similarly, the third row is shifted cyclically to the left by two bytes, and the fourth row is shifted cyclically to the left by three bytes.
– **MixColumns:** This is essentially a multiplication operation of the state matrix and fixed scalar matrix:

$$\begin{pmatrix} C_0 & C_4 & C_8 & C_{12} \\ C_1 & C_5 & C_9 & C_{13} \\ C_2 & C_6 & C_{10} & C_{14} \\ C_3 & C_7 & C_{11} & C_{15} \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} B_0 & B_4 & B_8 & B_{12} \\ B_5 & B_9 & B_{13} & B_1 \\ B_{10} & B_{14} & B_2 & B_6 \\ B_{15} & B_3 & B_7 & B_{11} \end{pmatrix}$$

– **AddRoundKey:** Key addition is very simple, it refers to performing bitwise XOR of state matrix C_i with the current round key. All round keys are generated in advance by the key schedule, and each round key is also 16 bytes (128 bits).

Notice that there is one more AddRoundKey operation before the first round starts. And in the last round, no MixColumns operation is performed.

4 Homomorphic Evaluation of AES

In the TFHE scheme, gate bootstrapping or functional bootstrapping implements the computation of the accumulator through the successive CMUX gates, that is to say, assuming that the dimension of TLWE ciphertext is n, then the computation cost of a gate bootstrapping is about n times that of external multiplication. Thus, we consider using a cheaper CMUX as basic gate, constructed by the external multiplication of TRGSW and TRLWE ciphertext, to evaluate the S-box. However, note that the output ciphertext form of external multiplication is TRLWE. When we move to the next round of evaluation of AES, we need TRGSW ciphertext as the selector ciphertext. Thus, in order to address this incompatibility, the circuit bootstrapping must be used to convert TLWE ciphertext to TRGSW ciphertext to maintain the operation of the whole circuit. Meanwhile, in order to efficiently evaluate the XOR operation in AddRoundKey, we choose to encode the message $\{0,1\}$ to $\{0,1/2\}$ over Torus, which makes XOR to be simple addition.

In [12], the authors give three packing techniques to optimize lookup table: horizontal packing, vertical packing and mixed packing. Next, in Sect. 4.1, we analyze the cost of their application to the S-box. Circuit bootstrapping, as a

bridge for ciphertext conversion, would be the bottleneck of the whole computation. We combine the recently proposed PBSmanyLUT technique [13] to reduce the internal computational cost in Sect. 4.2. Our full description of the AES evaluation is shown in Sect. 4.3.

4.1 Discussion on S-Box Lookup Table

For an 8-to-8-bit S-box lookup table, its output table size is 256×8. Specifically, in the parameter settings we assume the degree of the ring polynomial $N = 1024$ and give a customized lookup table optimization algorithm for the S-box. Next, we analyze the effect of different packing methods on S-box evaluation optimization. Notice that we now assume that the ciphertext form of all selector bits is TRGSW.

- Horizontal packing: We can pack each line of the output result as a TRLWE ciphertext, each S-box lookup table can get the lookup results at the same time using $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$ CMUX gates, located in the first 8 coefficients of the result TRLWE ciphertext, so we can reduce CMUX consumption by 8 times compared with no packing.
- Vertical packing: We can pack each column of the output result as a TRLWE sample, and then use the blind rotation algorithm for each TRLWE ciphertext to obtain the desired value, which is located at the constant term position of each resulting TRLWE ciphertext. Each blind rotation requires 8 CMUX gates, thus, we need $8 \times 8 = 64$ CMUX gates in total in one S-box evaluation, decreasing by a factor of 32 CMUX consumption compared with no packing.
- Mixed packing: here $2^8 \cdot 8 = 2 \cdot N = 2048$, we can combine horizontal and vertical packing to pack all the output results into two TRLWE ciphertexts. Firstly, we use the last one TRGSW ciphertext to pick out the target TRLWE through one CMUX gate, and then use the remaining 7 TRGSW samples to perform the blind rotation on the target TRLWE, the desired value would be located in the first 8 positions of the resulting TRLWE. In this way, we only need to use 8 CMUX gates to implement the evaluation of the S-box.

Based on the above analysis, we choose to use mixed packing to accelerate the evaluation of the S-box further. It is noted that to be compatible with other operations of AES, we just need to use the SampleExtract algorithm to obtain the corresponding TLWE without any cost. And finally, we use KeySwitch algorithm to switch the dimension of TLWE ciphertext. Our S-box lookup table combined with mixed packing is shown in Algorithm 1.

4.2 Optimization of Circuit Bootstrapping

Circuit bootstrapping, as a bridge, can convert TLWE to TRGSW ciphertext, connecting the entire leveled evaluation circuit. The authors of [12] observed that each line of TRGSW is TRLWE ciphertext encrypting the message $\mu \cdot s_i \cdot \frac{1}{\mathfrak{B}^j}$ for $1 \leq i \leq k+1$ and $1 \leq j \leq \ell$. Thus the core idea of circuit bootstrapping

Algorithm 1. LUTMixedPacking

Input: Eight TRGSW ciphertexts $C_0, \cdots, C_7$
Input: Two TRLWE ciphertexts used for packing S-box d_0, d_1
Output: Eight TLWE ciphertexts $c_0, \cdots, c_7$
1: ACC $\leftarrow$ CMUX(C_7, d_1, d_0)
2: **for** $i = 0$ to 6 **do**
3: ACC $\leftarrow$ CMUX$(C_i, X^{-8\cdot 2^i \pmod{2N}} \cdot \text{ACC}, \text{ACC})$
4: **end for**
5: **for** $i = 0$ to 7 **do**
6: $c_i' \leftarrow$ SampleExtract(ACC, i)
7: $c_i \leftarrow$ KeySwitch(c_i')
8: **end for**
9: **return** $c_0, \cdots, c_7$

is to reconstruct each TRLWE ciphertext in the TRGSW ciphertext using the TLWE ciphertext, which runs about 0.137 s for a 110-bit security parameter. Circuit bootstrapping algorithm is divided into two steps:

- The first step converts the TLWE encryption of μ into the TLWE ciphertexts $\mu \cdot \frac{1}{\mathfrak{B}^j}$ using the ℓ TFHE functional bootstrapping technique, which accounts for about 70% of the total time.
- The second step is to multiply the secret key s_i to the TLWE ciphertext obtained in the previous step using the $(k+1)\ell$ private key switching algorithm, which accounts for 30% of the total time.

It is worth noting that since the input TLWE ciphertext is the same in the first step, ℓ bootstrapping operations can be optimized by multi-value bootstrapping [8] or PBSmanyLUT technique [13]. In detail, multi-value bootstrapping supports the evaluation of multiple functions on the same input using only one blind rotation. Firstly, extract a common function v_0 from all functions TV_{F_i}, then use the input ciphertext to blindly rotate test polynomial v_0, finally TV_{F_i}/v_0(with small norm coefficient) is multiplied by accumulator ACC to obtain the result respectively. The PBSmanyLUT first uses the modulus switching to set some lowest bits of the phase to zero, and then uses just one blind rotation to obtain all the function values, it must be noted that we need to set the test polynomial accordingly. The resultant ciphertext produced by PBSmanyLUT has less noise compared with multi-value bootstrapping, so we use PBSmanyLUT to accelerate the first step of circuit bootstrapping.

Recall that our message space of TLWE is the entire Torus, however, the LUT evaluation can only compute polynomial functions with negacyclic property due to the fact that $X^{i+N} \equiv -X^i \bmod X^N + 1$. We set a new test polynomial that satisfies this property for PBSmanyLUT as follows:

$$P(X) = \sum_{i=0}^{\frac{N}{2^{\rho}\cdot 2}-1} \sum_{j=0}^{2^{\rho}-1} (-1) \cdot \frac{1}{2\mathfrak{B}^j} X^{2^{\rho}\cdot i+j} + \sum_{i=\frac{N}{2^{\rho}\cdot 2}}^{\frac{N}{2^{\rho}}-1} \sum_{j=0}^{2^{\rho}-1} \frac{1}{2\mathfrak{B}^j} X^{2^{\rho}\cdot i+j}$$

where $\rho = \lceil \log_2(\ell) \rceil$ and $\mathfrak{B}$ is the basis of gadget. Then, the ℓ independent bootstrappings in circuit bootstrapping could be replaced by:

$$\begin{cases} \{\mathrm{ct}_i\}_{i\in[1,\ell]} \leftarrow \mathbf{PBSmanyLUT}\left(\mathrm{ct}_{\mathrm{in}}, \mathrm{BSK}, P(x)\cdot X^{N/2^{\rho+1}}, 1, 0, \rho\right) \\ \forall i \in [1,\ell], \mathrm{ct}_i + \left(\mathbf{0}, \frac{1}{2\mathfrak{B}^i}\right) \end{cases}$$

For more details on **PBSmanyLUT** please refer to Algorithm 6 of [13], and the proof is the same as Lemma 4 of [13]. Our circuit bootstrapping combined with the PBSmanyLUT technique is presented in Algorithm 2.

Algorithm 2. CBwithPBSmanyLUT

Input: a test polynomial $P(X)$
Input: a level 0 TLWE ciphertext: ct_{in}
Input: a bootstrapping key from level 0 to level 2: BSK
Input: $k+1$ private keyswitch keys from level 2 to level 1: KS
Output: a level 1 TRGSW ciphertext C
1: $\{\mathrm{ct}_i\}_{i\in[1,\ell]} \leftarrow \mathbf{PBSmanyLUT}\left(\mathrm{ct}_{\mathrm{in}}, \mathrm{BSK}, P(X)\cdot X^{N/2^{\rho+1}}, 1, 0, \rho\right)$
2: **for** $w = 1$ to ℓ **do**
3: $\quad \mathrm{ct}'_w = \mathrm{ct}_w + \left(\mathbf{0}, \frac{1}{2\mathfrak{B}^w}\right)$
4: $\quad$ **for** $u = 1$ to $k+1$ **do**
5: $\quad\quad c^{(u,w)} = \text{Private KeySwitch}(KS, \mathrm{ct}'_w)$
6: $\quad$ **end for**
7: **end for**
8: **return** $C = (c^{u,w})_{1\le u\le k+1, 1\le w\le \ell}$

4.3 Full Description of the AES Evaluation

In the above two subsections, we have presented an efficient evaluation of SubBytes and an optimization regarding circuit bootstrapping. It is obvious that the ShiftRows operation is free and AddRoundKey is also almost free due to message encoding. The additional operation that needs to be handled is MixColumns. The MixColumns operation is a 4×4 matrix multiplication with constant terms $\{x+1, x, 1\}$ modulo $(x^8+x^4+x^3+x+1)$. Interestingly, this modular multiplication can be computed by simple XOR and shift operation [21], as shown below. Here we represent a byte from the lowest bit to the highest bit as $(b_0b_1b_2b_3b_4b_5b_6b_7)$.

$$b_0b_1b_2b_3b_4b_5b_6b_7 \xrightarrow{\times 1} b_0b_1b_2b_3b_4b_5b_6b_7$$

$$b_0b_1b_2b_3b_4b_5b_6b_7 \xrightarrow{\times x} b_7b_0b_1b_2b_3b_4b_5b_6 \oplus 0b_70b_7b_7000$$

$$b_0b_1b_2b_3b_4b_5b_6b_7 \xrightarrow{\times (x+1)} b_0b_1b_2b_3b_4b_5b_6b_7 \oplus b_7b_0b_1b_2b_3b_4b_5b_6 \oplus 0b_70b_7b_7000$$

In this way, we can convert all the multiplication operations in MixColumns into bitwise XOR operation and simple shift, which is exactly what we want.

Thus, ShiftRows, AddRoundKey and MixColumns can be performed at Level 0, while SubBytes would be performed in Level 1. Our evaluation framework[1] is clearly presented in Fig. 1, the details of the corresponding algorithm are given in Appendix A.

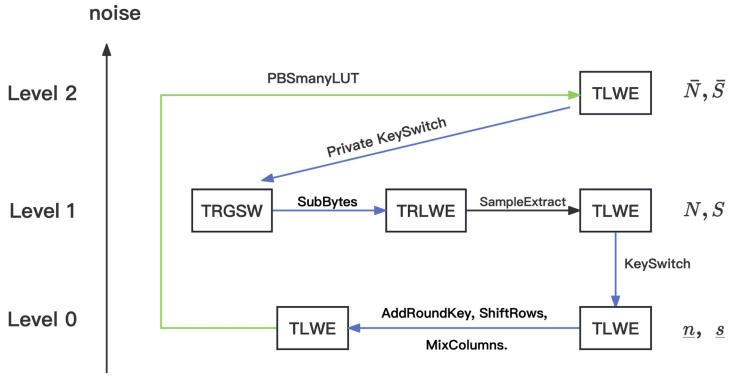


Fig. 1. Fregata: the homomorphic evaluation framework of AES performed at different levels.

Noise Analysis. The circuit bootstrapping algorithm crosses 3 levels, each involving different ciphertext types with their own parameter settings and associated keys. The main difference between the three encryption levels is the amount of noise supported, the higher the level, the lower the noise. Level 0 corresponds to small parameters, large noise, instantaneous computation, but only very limited linear operations can be tolerated.

Indeed, the first step of the circuit bootstrapping, that is, the PBSmanyLUT, plays the role of refreshing noise throughout the evaluation process. Now we analyze the internal noise variance of the ciphertext from the PBSmanyLUT to the end in each round.

The homomorphic evaluation of S-box in Fig. 1 contains five homomorphic operations: PBSmanyLUT, Private KeySwitch, Sbox lookup table, SampleExtract and Public KeySwitch, where the SampleExtract operation does not introduce any noise and Public KeySwitch is utilized to adjust the parameters from (N, S) to $(\underline{n}, \underline{s})$. Thus, we can calculate the error after Public KeySwitch on Level 0 by analyzing the error variance of each step separately. Firstly, according to Theorem 4 in [13], we have

[1] We called our framework "Fregata", which cleverly reads like "Free Gate", emphasizing the fact that it eliminates the cost for XOR gate due to message encoding. Its speed reflects our efficient homomorphic evaluation of S-box.

$$Var(PBSmanyLUT) \leq \underline{n}\bar{\ell}(k+1)\bar{N}\frac{\bar{B_g}^2+2}{12}Var(BSK) + \underline{n}\frac{q^2-\bar{B_g}^{2\bar{\ell}}}{24\bar{B_g}^{2\bar{\ell}}}(1+\frac{k\bar{N}}{2}) + \frac{\underline{n}k\bar{N}}{32} + \frac{\underline{n}}{16}(1-\frac{k\bar{N}}{2})^2$$

After performing Private KeySwitch, by Theorem 2.7 in [12], we have

$$Var(TRGSW) \leq Var(PBSmanyLUT) + \bar{t}\bar{N}\bar{B_{ks}}^2 Var(privateKS)$$

In order to evaluate the Sbox, we use 8 CMUX gates to lookup table and then perform SampleExtract and KeySwitch in Algorithm 1, we have

$$Var(\text{TLWE}_{\text{Level0}}) \leq 8 \cdot ((k+1)\ell N\beta^2 Var(TRGSW)) + Var(TRLWE_{SBox}) + \underline{n}tNB_{ks}^2 Var(KS)$$

where $Var(BSK), Var(privateKS), Var(TRLWE_{Sbox})$ and $Var(KS)$ can be found in [12].

5 Scalability: Homomorphic Evaluation of SM4

In this section, we will show that our evaluation framework can be also extended to the homomorphic evaluation of other ciphers, such as symmetric encryption scheme SM4. SM4 is a Chinese block cipher standard used for protecting wireless networks and was released publicly in 2006 [17]. Now it has become the international standard of ISO/IEC, which effectively promotes the improvement of ISO/IEC symmetric cryptographic algorithms. The structure of SM4 is similar to the AES algorithm, but it uses generalized Feistel structure. And its encryption computation requires up to 32 rounds, where each round contains four parts: key addition, S-box, linear transformation and XOR operation as follows. Let the plaintext inputs be $(X_0, X_1, X_2, X_3) \in (\mathbb{Z}_2^{32})^4$ and round keys be $rk_i \in \mathbb{Z}_2^{32} (i = 0, 1, 2, \cdots, 31)$, which are derived from the initial key through the key expansion.

- Key addition: the 32-bit output word $A = (a_0, a_1, a_2, a_3) \in (\mathbb{Z}_2^8)^4 = (X_{i+1} \oplus X_{i+2} \oplus X_{i+3} \oplus rk_i)$;
- Non-linear substitution(S-box) τ: the 32-bit output word $B = (b_0, b_1, b_2, b_3) \in (\mathbb{Z}_2^8)^4$ is derived by $(S(a_0), S(a_1), S(a_2), S(a_3))$;
- Linear transformation: the 32-bit output word $C = L(B) = B \oplus (B \lll_{32} 2) \oplus (B \lll_{32} 10) \oplus (B \lll_{32} 18) \oplus (B \lll_{32} 24)$;
- XOR operation: $X_{i+4} = C \oplus X_i$.

After 32 rounds the final encryption result is $(X_{35}, X_{34}, X_{33}, X_{32})$. Xue [40] presented the homomorphic evaluation of SM4 based on the BGV scheme using Gentry's method [26]. However, since the number of rounds in SM4 is more than

three times that of AES, the evaluation of SM4 must use large encryption parameters to match the multiplicative depth, resulting extremely long latency. To be specific, it takes up to 6 h and 1.5 h in leveled and bootstrapped implementation version.

Next, we give an efficient evaluation of SM4 based on our evaluation framework. Notice that in the homomorphic SM4 evaluation, the results of the previous few rounds are superimposed on the XOR operation and key addition of next round. To be precise, during the final XOR operation of each round, the noise of ciphertext C would be superimposed on the noise of ciphertext X_i from the previous round in the encrypted domain. As a result, the noise magnitude of the ciphertext generated by the final XOR operation increases with each round of evaluation. Moreover, three noisy TLWE ciphertexts and a fresh TLWE ciphertext of round key will participate in key addition of the next round. Therefore, intuitively, after several rounds of evaluation, the TLWE ciphertexts generated by the key addition operation will be extremely noisy, leading to decryption failure.

It is a bit tricky to actually determine at which round the decryption would fail. We experimentally verify that the TLWE ciphertext generated by key addition after eight rounds of evaluation will fail to decrypt with a high probability due to excessive noise. Therefore, we fix this issue by introducing the identity bootstrapping (more details about identity bootstrapping algorithm, please see Algorithm 3 of [11]) after each round of evaluation. In this way, we can avoid the problem of excessive noise due to key addition.

Finally, we redesign the homomorphic evaluation flow of SM4 by introducing identity bootstrapping on level 0, all homomorphic operations performed at different levels are presented in Fig. 2.

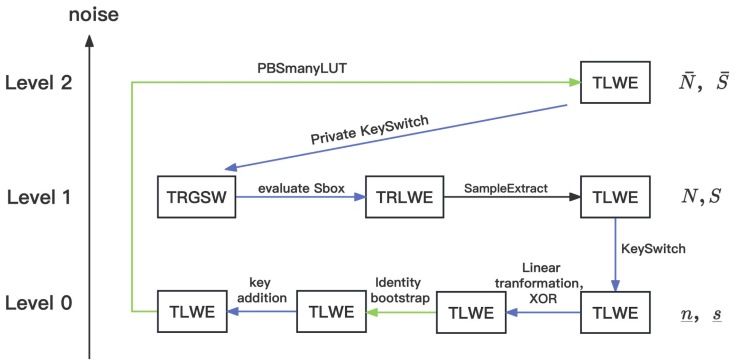


Fig. 2. Diagram of the homomorphic evaluation of SM4 performed at different levels.

6 Implementation and Performance Results

In this section, we implement our proposed framwork to test the accuracy and efficiency of evaluating AES and SM4 based on the TFHE scheme. All the experiments were conducted on a single core of 12th Gen Intel(R) Core(TM) i5-12500 × 12 with 15.3 GB RAM, running the Ubuntu 20.04 operating system.

6.1 Parameters Selection.

Note that the original implementation of circuit bootstrapping[2] is not compatible with the TFHE library[3]. TFHEpp [34] is full Scracthed pure C++ version of TFHE, which supports circuit bootstrapping and private PBSmanyLUT [13]. Therefore, we implement the homomorphic evaluation of SM4 and AES in the TFHEpp library. Circuit bootstrapping crosses 3 levels, each involving different ciphertext types with their own parameter settings and associated keys. Specifically, the relevant parameters of the three levels are shown in Table 1. All of our parameters are estimated to be above $\lambda = 128$ bit security according to the latest iteration of the LWE estimator[4].

Table 1. Parameters for levels 0, 1 and 2 in circuit bootstrapping mode

Level	dimension of the TLWE	noise stdev	decomposition basis	decomposition length
0	$\underline{n} = 635$	$\underline{\alpha} = 2^{-15}$	–	–
1	$N = 1024$	$\alpha = 2^{-25}$	$B_g = 2^6$	$\ell = 3$
2	$\bar{N} = 2048$	$\bar{\alpha} = 2^{-44}$	$\bar{B_g} = 2^9$	$\bar{\ell} = 4$
$1 \to 0$	–	–	$B_{ks} = 2^2$	$t = 7$
$2 \to 1$	–	–	$\bar{B_{ks}} = 2^3$	$\bar{t} = 10$

6.2 Performance and Analysis

Our implementation focuses on testing the latency time of the evaluation of a single block. In order to demonstrate the efficiency of our evaluation method as much as possible, we also evaluate the AES based on trivial gate bootstrapping mode, where all computational units of AES are replaced by bootsXOR and bootsMUX gates in TFHE. Meanwhile, for SM4, since the numbers of shift bits are exactly a multiple of 2 in the linear transformation, we choose to use two-bit wise encryption and provide efficient implementation using functional

[2] https://github.com/tfhe/experimental-tfhe.
[3] https://github.com/tfhe/tfhe.
[4] https://bitbucket.org/malb/lwe-estimator.

bootstrapping. If we adopt larger-bit (> 2) wise encryption, we would need to extract the ciphertext of the corresponding bit from the ciphertext to perform the shift operation of linear transformation, which is an extremely costly operation [32]. The experiments are run 1000 times on a single core of our machine, and we take the average time as running latency time. The specific implementation results are presented in Table 2.

Table 2. Our execution times compared to the state of the art using just one thread (The data with $*$ refers to 16 threads).

scheme	mode	AES	SM4
BGV	leveled	Gentry et al. [26] 4 mins	Xue [40] 6 h
	bootstrapped	[26] 18 mins	[40] 1.58 h
TFHE	gate bootstrapping	1.43 h	1.68 h
	functional bootstrapping	Stracovsky et al. [38] 4.2 mins*	
		Trama et al. [39] 4.5 mins	6.1 mins
	circuit bootstrapping	**86** s	**78** s

It can be seen that our novel evaluation based on circuit bootstrapping mode achieves the best latency performance. Specifically, the homomorphic AES evaluation based on circuit bootstrapping mode takes 86 s, which is about 3 times faster than the state-of-the-art [39]. The circuit bootstrapping mode takes 78 s to perform SM4 with one CPU thread, which is up to 73× reduction in latency compared with Xue [40] and 4.7 × reduction compared with functional bootstrapping mode. Considering that our implementation uses only one CPU thread, using the OpenMP library can offer more speedups for our implementation since circuit bootstrappings each round are performed in parallel in our evaluation framework. For example, we expect to achieve an evaluation of AES in about 9 s if we use the "i7-server" as described in Trama et al. [39].

Although the number of encryption rounds of AES is only 10, less than SM4, it requires $16 \times 10 = 160$ S-box LUTs. The number of rounds of SM4 is up to 32, with only 4 paralleled LUTs each round, requiring a total of $32 \times 4 = 128$ LUTs. Moreover, the main consumption time of our evaluation mode comes from circuit bootstrapping before the S-box, resulting in the homomorphic evaluation of AES being slower than SM4. Table 3 and Table 4 detail the time portion of each part of the AES and SM4 evaluation based on circuit bootstrapping, respectively. Notice that the circuit bootstrapping becomes the most expensive part of the computation, while the S-box lookup table takes about only 16 ms, which is almost negligible.

Table 3. The time spent for each core operation of AES evaluation and their percentage relative to the total time in circuit bootstrapping mode.

Operation	Circuit Boot	SubBytes	Identity KS	Total time
Time (Ratio)	84.48 s (98.31%)	18.48 ms (0.02%)	1.42 s (1.65%)	**86 s**

Table 4. The time spent for each core operation of SM4 evaluation and their percentage relative to the total time in circuit bootstrapping mode. Identity KS denotes KeySwitch after SampleExtract and Identity Boot represents Identity bootstrapping.

Operation	Circuit Boot	S-box LUT	Identity KS	Identity Boot	Total time
Time (Ratio)	68.11 s (87.66%)	15.59 ms (0.02%)	1.43 s (1.83%)	8.1 s (10.41%)	**78 s**

7 Conclusion

In this paper, we propose a novel evaluation framework to achieve faster AES evaluation based on the circuit bootstrapping of TFHE. We utilize techniques such as hybrid packing and optimized circuit bootstrapping to achieve the optimal latency of current AES homomorphic evaluation, which is about 3 times faster than the state-of-the-art implementation. Meanwhile, our framework can be extended to other Feistel structured ciphers, for example, we achieve efficient evaluation of the block cipher SM4, which is 73× faster than the current best publicly available implementation. Although TFHE-based evaluation is far inferior to BGV-based packing method in terms of amortization rate, our framework combined with multi-thread would hopefully approach the amortization time further.

Improving the efficiency of the circuit bootstrapping would be our next research direction, especially the optimization of private KeySwitching. We hope that our work could provide a guide for designing new FHE-friendly symmetric encryption algorithms.

Acknowledgement. We thank the anonymous ISC2023 reviewers for their helpful comments. This work was supported by the Huawei Technologies Co., Ltd. and CAS Project for Young Scientists in Basic Research Grant No. YSBR-035.

A Detailed Algorithms

Algorithm 3. Homomorphic evaluation of AES via circuit bootstrapping

Input: two TRLWE ciphertexts used for packing AES S-box: d_0, d_1
Input: the TLWE ciphertext of the $(16 * i + j)$-th input bit $C_{i,j}^{input_tlwe}$, where $0 \leq i \leq 15, 0 \leq j \leq 7$;
Input: the TLWE ciphertext of the $(16 * i + j)$-th bit of the r-th round key $C_{r,i,j}^{rk_tlwe}$, where $0 \leq r \leq 10, 0 \leq i \leq 15, 0 \leq j \leq 7$
Output: C^{output_tlwe}: double-encrypted ciphertext $Enc^{HE}\left(Enc^{AES}\left(m, rk\right)\right)$

// AddRoundKey
$C^{input_tlwe} = C^{input_tlwe} + C_0^{rk_tlwe}$
for $r = 1$ to 9 **do**
  // Circuit bootstrapping and SubBytes
  for $i = 0$ to 15 **do**
    for $j = 0$ to 7 **do**
      $C_{i,j}^{trgsw} = CBwithPBSmanyLUT(C_{i,j}^{input_tlwe})$
    end for
    $C_i^{input_tlwe} = LUTMixedPacking(C_i^{trgsw}, d_0, d_1)$
  end for
  // ShiftRows
  $C^{input_tlwe} = \text{CipherShiftRows}(C^{input_tlwe})$
  // Mixcolums
  $C^{input_tlwe} = \text{CipherMixcolums}(C^{input_tlwe})$
  // AddRoundKey
  $C^{input_tlwe} = C^{input_tlwe} + C_r^{rk_tlwe}$
end for
// Circuit bootstrapping and SubBytes
for $i = 0$ to 15 **do**
  for $j = 0$ to 7 **do**
    $C_{i,j}^{trgsw} = CBwithPBSmanyLUT(C_{i,j}^{input_tlwe})$
  end for
  $C_i^{output_tlwe} = LUTMixedPacking(C_i^{trgsw}, d_0, d_1)$
end for
// ShiftRows
$C^{output_tlwe} = \text{CipherShiftRows}(C^{output_tlwe})$
// AddRoundKey
$C^{output_tlwe} = C^{output_tlwe} + C_{10}^{rk_tlwe}$
return C^{output_tlwe}

Algorithm 4. CipherShiftsRows

Input: $C_{i,j}^{input_tlwe}$, $0 \leq i \leq 15, 0 \leq j \leq 7$
Output: $C_{i,j}^{output_tlwe}$, $0 \leq i \leq 15, 0 \leq j \leq 7$
for $i = 0$ to 15 **do**
  $C_i^{output_tlwe} = C_{5i \pmod{16}}^{input_tlwe}$
end for
return C^{output_tlwe}

Algorithm 5. CipherMixColums

Input: $C_{i,j}^{input_tlwe}$, $0 \leq i \leq 15, 0 \leq j \leq 7$
Output: $C_{i,j}^{output_tlwe}$, $0 \leq i \leq 15, 0 \leq j \leq 7$
for $i = 0$ to 3 **do**
  $Tmp = C_{4i}^{input_tlwe} + C_{4i+1}^{input_tlwe} + C_{4i+2}^{input_tlwe} + C_{4i+3}^{input_tlwe}$
  $Tm = C_{4i}^{input_tlwe} + C_{4i+1}^{input_tlwe}$
  $Tm = \text{CipherMul2}(Tm)$
  $C_{4i}^{output_tlwe} = C_{4i}^{input_tlwe} + Tm + Tmp$
  $Tm = C_{4i+1}^{input_tlwe} + C_{4i+2}^{input_tlwe}$
  $Tm = \text{CipherMul2}(Tm)$
  $C_{4i+1}^{output_tlwe} = C_{4i+1}^{input_tlwe} + Tm + Tmp$
  $Tm = C_{4i+2}^{input_tlwe} + C_{4i+3}^{input_tlwe}$
  $Tm = \text{CipherMul2}(Tm)$
  $C_{4i+2}^{output_tlwe} = C_{4i+2}^{input_tlwe} + Tm + Tmp$
  $Tm = C_{4i+3}^{input_tlwe} + C_{4i}^{input_tlwe}$
  $Tm = \text{CipherMul2}(Tm)$
  $C_{4i+3}^{output_tlwe} = C_{4i+3}^{input_tlwe} + Tm + Tmp$
end for
return C^{output_tlwe}

Algorithm 6. CipherMul2

Input: $C_i^{input_tlwe}$, $0 \leq i \leq 7$
Output: $C_i^{output_tlwe}$, $0 \leq i \leq 7$
$C_0^{output_tlwe} = C_7^{input_tlwe}$
$C_1^{output_tlwe} = C_0^{input_tlwe} + C_7^{input_tlwe}$
$C_2^{output_tlwe} = C_1^{input_tlwe}$
$C_3^{output_tlwe} = C_2^{input_tlwe} + C_7^{input_tlwe}$
$C_4^{output_tlwe} = C_3^{input_tlwe} + C_7^{input_tlwe}$
$C_5^{output_tlwe} = C_4^{input_tlwe}$
$C_6^{output_tlwe} = C_5^{input_tlwe}$
$C_7^{output_tlwe} = C_6^{input_tlwe}$
return C^{output_tlwe}

References

1. Albrecht, M.R., Rechberger, C., Schneider, T., Tiessen, T., Zohner, M.: Ciphers for MPC and FHE. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015. LNCS, vol. 9056, pp. 430–454. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46800-5_17
2. Ashur, T., Mahzoun, M., Toprakhisar, D.: Chaghri - a FHE-friendly block cipher. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, pp. 139–150. ACM (2022)
3. Bonte, C., Iliashenko, I., Park, J., Pereira, H.V.L., Smart, N.P.: FINAL: faster FHE instantiated with NTRU and LWE. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022. LNCS, vol. 13792, pp. 188–215. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-22966-4_7

4. Boura, C., Gama, N., Georgieva, M., Jetchev, D.: Simulating homomorphic evaluation of deep learning predictions. In: Dolev, S., Hendler, D., Lodha, S., Yung, M. (eds.) CSCML 2019. LNCS, vol. 11527, pp. 212–230. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-20951-3_20
5. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 868–886. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-32009-5_50
6. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Innovations in Theoretical Computer Science 2012, pp. 309–325. ACM, New York (2012)
7. Canteaut, A., et al.: Stream ciphers: a practical solution for efficient homomorphic-ciphertext compression. J. Cryptol. **31**(3), 885–916 (2018). https://doi.org/10.1007/s00145-017-9273-9
8. Carpov, S., Izabachène, M., Mollimard, V.: New techniques for multi-value input homomorphic evaluation and applications. In: Matsui, M. (ed.) CT-RSA 2019. LNCS, vol. 11405, pp. 106–126. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-12612-4_6
9. Cheon, J.H., et al.: Batch fully homomorphic encryption over the integers. In: Johansson, T., Nguyen, P.Q. (eds.) EUROCRYPT 2013. LNCS, vol. 7881, pp. 315–335. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-38348-9_20
10. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017. LNCS, vol. 10624, pp. 409–437. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-70694-8_15
11. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: bootstrapping in less than 0.1 seconds. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016. LNCS, vol. 10031, pp. 3–33. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53887-6_1
12. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster packed homomorphic operations and efficient circuit bootstrapping for TFHE. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017. LNCS, vol. 10624, pp. 377–408. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-70694-8_14
13. Chillotti, I., Ligier, D., Orfila, J.-B., Tap, S.: Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021. LNCS, vol. 13092, pp. 670–699. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-92078-4_23
14. Cid, C., Indrøy, J.P., Raddum, H.: FASTA – a stream cipher for fast FHE evaluation. In: Galbraith, S.D. (ed.) CT-RSA 2022. LNCS, vol. 13161, pp. 451–483. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-95312-6_19
15. Coron, J.-S., Lepoint, T., Tibouchi, M.: Scale-invariant fully homomorphic encryption over the integers. In: Krawczyk, H. (ed.) PKC 2014. LNCS, vol. 8383, pp. 311–328. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-642-54631-0_18
16. Cosseron, O., Hoffmann, C., Méaux, P., Standaert, F.: Towards case-optimized hybrid homomorphic encryption - featuring the elisabeth stream cipher. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022. LNCS, vol. 13793, pp. 32–67. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-22969-5_2
17. Diffie, W., Ledin, G.: SMS4 encryption algorithm for wireless networks. Cryptology ePrint Archive, Report 2008/329 (2008). https://eprint.iacr.org/2008/329

18. van Dijk, M., Gentry, C., Halevi, S., Vaikuntanathan, V.: Fully homomorphic encryption over the integers. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 24–43. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-13190-5_2
19. Dobraunig, C., et al.: Rasta: a cipher with low ANDdepth and few ANDs per bit. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018. LNCS, vol. 10991, pp. 662–692. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-96884-1_22
20. Dobraunig, C., Grassi, L., Helminger, L., Rechberger, C., Schofnegger, M., Walch, R.: Pasta: a case for hybrid homomorphic encryption. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2023**(3), 30–73 (2023). https://doi.org/10.46586/tches.v2023.i3.30-73
21. Doröz, Y., Hu, Y., Sunar, B.: Homomorphic AES evaluation using the modified LTV scheme. Des. Codes Cryptogr. **80**, 333–358 (2016). https://doi.org/10.1007/s10623-015-0095-1
22. Doröz, Y., Shahverdi, A., Eisenbarth, T., Sunar, B.: Toward practical homomorphic evaluation of block ciphers using prince. In: Böhme, R., Brenner, M., Moore, T., Smith, M. (eds.) FC 2014. LNCS, vol. 8438, pp. 208–220. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-662-44774-1_17
23. Ducas, L., Micciancio, D.: FHEW: bootstrapping homomorphic encryption in less than a second. In: Oswald, E., Fischlin, M. (eds.) EUROCRYPT 2015. LNCS, vol. 9056, pp. 617–640. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46800-5_24
24. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144 (2012). https://eprint.iacr.org/2012/144
25. Gentry, C.: A fully homomorphic encryption scheme (2009)
26. Gentry, C., Halevi, S., Smart, N.P.: Homomorphic evaluation of the AES circuit. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 850–867. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-32009-5_49
27. Ha, J., et al.: Masta: an HE-friendly cipher using modular arithmetic. IEEE Access **8**, 194741–194751 (2020)
28. Ha, J., Kim, S., Lee, B., Lee, J., Son, M.: Rubato: noisy ciphers for approximate homomorphic encryption. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022. LNCS, vol. 13275, pp. 581–610. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-06944-4_20
29. Hebborn, P., Leander, G.: Dasta - alternative linear layer for rasta. IACR Trans. Symmetric Cryptol. **2020**(3), 46–86 (2020)
30. Lee, Y., et al.: Efficient FHEW bootstrapping with small evaluation keys, and applications to threshold homomorphic encryption. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023. LNCS, vol. 14006, pp. 227–256. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30620-4_8
31. Lepoint, T., Naehrig, M.: A comparison of the homomorphic encryption schemes FV and YASHE. In: Pointcheval, D., Vergnaud, D. (eds.) AFRICACRYPT 2014. LNCS, vol. 8469, pp. 318–335. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-06734-6_20
32. Liu, Z., Micciancio, D., Polyakov, Y.: Large-precision homomorphic sign evaluation using FHEW/TFHE bootstrapping. IACR Cryptology ePrint Archive, p. 1337 (2021). https://eprint.iacr.org/2021/1337
33. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Gilbert, H. (ed.) EUROCRYPT 2010. LNCS, vol. 6110, pp. 1–23. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-13190-5_1

34. Matsuoka, K.: TFHEpp: pure C++ implementation of TFHE cryptosystem (2020). https://github.com/virtualsecureplatform/TFHEpp
35. Méaux, P., Journault, A., Standaert, F.-X., Carlet, C.: Towards stream ciphers for efficient FHE with low-noise ciphertexts. In: Fischlin, M., Coron, J.-S. (eds.) EUROCRYPT 2016. LNCS, vol. 9665, pp. 311–343. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-49890-3_13
36. Naehrig, M., Lauter, K.E., Vaikuntanathan, V.: Can homomorphic encryption be practical? In: Proceedings of the 3rd ACM Cloud Computing Security Workshop, CCSW, pp. 113–124. ACM, New York (2011)
37. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. J. ACM **56**(6), 34:1–34:40 (2009)
38. Stracovsky, R., Mahdavi, R.A., Kerschbaum, F.: Faster evaluation of AES using TFHE. Poster Session, FHE.Org - 2022 (2022). https://rasoulam.github.io/data/poster-aes-tfhe.pdf
39. Trama, D., Clet, P., Boudguiga, A., Sirdey, R.: At last! A homomorphic AES evaluation in less than 30 seconds by means of TFHE. IACR Cryptology ePrint Archive, p. 1020 (2023). https://eprint.iacr.org/2023/1020
40. Xue, Y.: Homomorphic evaluation of the SM4. IACR Cryptology ePrint Archive, p. 1340 (2020). https://eprint.iacr.org/2020/1340